

EXPERIMENT -5

Student Name: Kapish Gupta

Branch: AIT-CSE (AIML)

Semester: 5

Subject Name: Database Management System

UID: 24BAI70577

Section/Group: 24AIT_KRG-1A

Subject Code: 24CSH-298

Q1) A fruit distribution company records the number of apples and oranges sold each day. For every date, exactly one record exists for apples and one record exists for oranges.

Generate a daily sales comparison report showing the difference between the number of apples sold and the number of oranges sold on the same day. The difference should be calculated as:

Difference = Apples Sold - Oranges Sold

Display the sale date and the calculated difference. The result should be sorted by sale date in ascending order.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

EXPERIMENT -5

```
SELECT sale_date , SUM(  
    CASE  
        WHEN fruit = 'apples' THEN sold_num  
        ELSE -sold_num  
    END  
) SALE_DATA  
FROM Sales  
GROUP BY sale_date  
ORDER BY SALE_DATA DESC
```

OUTPUT SCREENSHOTS:

Output:

CREATE TABLE

INSERT 0 8

sale_date	sale_data
2020-05-03	20
2020-05-01	2
2020-05-02	0
2020-05-04	-1

(4 rows)

EXPERIMENT -5

LEARNING OUTCOMES:

- ☐ **Understand SQL JOINS**
 - Use LEFT JOIN to keep all registered users, even if they have no confirmation records.
- ☐ **Use Aggregate Functions**
 - Use COUNT() to calculate:
 - Total confirmation requests
 - Successful (confirmed) confirmations
- ☐ **Conditional Aggregation**
 - Use CASE WHEN inside COUNT() or SUM() to count only successful confirmations.
- ☐ **Handle Missing Data**
 - Understand that a user with no confirmation request should have a confirmation rate of 0, not NULL.

CONCLUSION:

The problem demonstrates how SQL can be used to calculate a user's confirmation rate by combining data from multiple tables. By using LEFT JOIN, conditional aggregation, GROUP BY, COALESCE(), and ROUND(), we can correctly calculate the rate for every registered user, including users who have no confirmation requests. This strengthens understanding of SQL joins, aggregate functions, conditional counting, NULL handling, and data aggregation.

EXPERIMENT -5

(B) Problem Statement

A company stores hierarchical employee reporting information in a table. Each record represents a node in a hierarchy where:

- **id** represents the current node.
- **p_id** represents the parent node.

Classify every node into one of the following categories:

- **Root:** A node that does not have any parent.
- **Leaf:** A node that has a parent but does not have any children.
- **Inner:** A node that has both a parent and at least one child.

Display the node ID along with its category. Return the result sorted by node ID.

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
SELECT
    t.id,
    CASE
        WHEN t.p_id IS NULL THEN 'Root'
        WHEN EXISTS (
            SELECT 1
            FROM Tree c
            WHERE c.p_id = t.id
        ) THEN 'Inner'
        ELSE 'Leaf'
    END AS category
FROM Tree t
ORDER BY t.id;
```

EXPERIMENT -5

OUTPUT SCREENSHOTS:

	id [PK] integer	category text
1	1	Root
2	2	Inner
3	3	Leaf
4	4	Leaf
5	5	Leaf

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use JOIN to combine data from multiple tables.
- Learn how to use the AVG() aggregate function to calculate average marks.
- Apply GROUP BY to organize data department-wise.
- Use ORDER BY to sort the results from highest to lowest average marks.
- Generate a meaningful analytical report by integrating student, department, subject, and marks data.

CONCLUSION:

The PostgreSQL query successfully generated a department-wise performance report by joining the relevant tables, calculating the average marks using the **AVG()** function, grouping the data by department, and sorting the results in descending order. This experiment